

트리 기반 모델: 결정트리에서 GBDT까지

결정 트리 · 지니불순도와 정보이득 · 랜덤 포레스트 · GBDT와 SHAP

Machine Learning 1 · 7주차

한국과학영재학교 (KSA)

Chapter 7

이번 주차 개요

이 챕터를 마치면

- 결정 트리의 구조를 설명하고, 지니불순도 감소량(정보이득)을 계산해 “좋은 질문”을 고른다
 - 랜덤 포레스트의 두 가지 무작위성이 분산을 줄이는 이유를 설명하고, OOB 점수로 검증 세트 없이 일반화 성능을 예측한다
 - 랜덤 포레스트(독립적·다수결)와 GBDT(순차적·잔차 추격)를 구분하고, GBDT의 트리 개수는 early stopping으로 정해야 하는 이유를 설명한다
 - SHAP으로 예측 하나를 특징별 기여로 분해하고, 모든 SHAP값의 합이 정확히 예측값을 재현하는 가산성을 확인한다
-
- **7.1** — 트리를 “질문”으로 세우고, 지니불순도·정보이득·U자 곡선을 다룬다
 - **7.2** — 트리를 **병렬**로 합치는 랜덤 포레스트(독립 + 다수결)
 - **7.3** — 트리를 **순서**로 합치는 GBDT(잔차를 고쳐가는 보정)와 그 설명(SHAP)

세 블록을 관통하는 줄기

합치는 방식이 만든다. 독립적으로 합치면(랜덤 포레스트) 흔들림만 줄고, 순차적으로 합치면(GBDT) 정확도는 더 오르지만 과적합의 위험도 함께 온다. 정확도의 대가로 잃은 “읽을 수 있음”은 SHAP(게임이론)으로 되찾는다.

7.1 결정 트리: 구조, 지니불순도, 정보이득

스무고개에서 결정 트리

“동물인가요? 네 다리로 걷나요? 야옹 소리를 내나요?” — **스무고개(20 Questions)**는 예/아니오 질문을 잘 골라 20번 안에 답을 좁혀내는 게임이다.

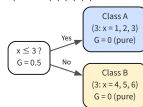
- **결정 트리**(Decision Tree) = 이 게임을 **알고리즘**으로 그대로 옮긴 것
- “좋은 질문”을 고르는 기준(지니불순도 감소량)을 수학으로 정의하면, 컴퓨터가 **스스로** 가장 잘 좁히는 질문을 찾아 나무 형태로 쌓는다
- 은행은 대출 거절 시 법(ECOA)에 따라 **구체적 사유**를 알려줘야 한다 — 트리 하나는 경로를 읽으면 되지만, GBDT는 “왜?”에 답하기 훨씬 어렵다

질문을 얼마나 “잘 좁히는가”를 숫자로 재는 기준은 무엇인가?

결정 트리의 구조: 이진나무

- **내부 노드** = 하나의 질문 (예: “ $x_2 > 5?$ ”)
- **리프**(leaf) 노드 = **예측값** (분류: 클래스, 회귀: 숫자)
- 예측 = 루트에서 시작해 예/아니오에 따라 **아래로 내려가 리프에 닿을 때까지** 경로를 따르는 것뿐
- 예: 6개 샘플 $x = [1, 2, 3, 4, 5, 6]$, 라벨 $[A, A, A, B, B, B]$ 를 “ $x \leq 3?$ ” **한 개의 질문**만으로 완벽 분류

Decision tree grown on 6 examples ($x = 1-6$, labels A,A,A,B,B,B) — internal nodes are questions, leaves are predictions



루트 “ $x \leq 3?$ ” — 예(1,2,3)→ 클래스 A,
아니오(4,5,6)→ 클래스 B. 두 리프 모두
순수($G = 0$).

경로 = 설명: 결정 트리의 강점

- 새 샘플 들어오면: (1) 루트에서 질문을 평가 → (2) 예/아니오에 따라 좌·우 자식으로 이동 → (3) 리프면 그 값을 내놓고 끝
- 이 **경로 자체가** “왜 이렇게 예측했는가”의 설명이다
- 대출 심사 예: “소득 < 3000만원 → 연령 > 45세 → **거절**”이라는 경로를 **그대로 읽어주면** 됨

설명가능성(interpretability)

루트에서 리프까지의 질문 경로를 사람이 읽을 수 있다. 이 장(7.3)에서는 **수백 개의 트리**가 합쳐지면 이 “경로 = 설명” 특성을 잃는 대가를 다시 논한다(SHAP).

왜 질문은 언제나 “축에 평행”한가

- 질문은 **특징 하나**만 쓴다: “ $x_j > t$?” 의 형태
- 여러 특징을 섞은 “ $x_1 + x_2 > 5$?” 같은 **대각선 경계**는 한 번에 만들 수 없다
- 그래서 결정 경계는 축에 평행한 직선 조각이 이어진 **계단**(staircase)으로밖에 나오지 않는다
- 이 제약은 **강점**(가독성, 스케일 불변)이자 동시에 **약점**(대각선 경계에 서툰다)

7.2절에서 “트리 하나는 계단, 숲은 부드러운 곡선”을 볼 때 이 계단이 재등장한다.

역사: 스무고개에서 CART까지

“질문을 잘 골라 답을 좁혀간다”는 아이디어는 1960~80년대에 통계학과 인공지능이 겹치는 장소에서 여러 이름으로 태어났다.

시기	이름	내용
1960s	CHAID	마케팅 연구: 고객을 “질문으로 쪼개는” 분할
1975	TDIDT	Quinlan: 예·아니오 질문 트리로 분류 문제를 푼다는 틀의 정식화
1984	CART	Breiman 외: 회귀를 같은 틀로 확장, 지니불순도 (분류)·제곱오차 감소(회귀)로 분기 기준 표준화
1986	ID3	Quinlan:

지니불순도: 노드가 얼마나 “섞여” 있나

- 스무고개를 잘하는 사람은 “생물인가요?”처럼 **반반으로 갈릴 만한** 질문을 먼저 던진다
- 그래서: “노드가 얼마나 섞여 있는가(불순도)”를 재고, **나누기 전 불순도 – 나눈 뒤 불순도**가 클수록 좋은 질문

지니불순도(Gini Impurity): 클래스 K 개, 클래스 k 의 비율이 p_k 일 때

$$G = 1 - \sum_{k=1}^K p_k^2$$

- 한 클래스로 **순수**하면($p_k = 1$): $G = 0$ — 가장 좋은 상태
- 반반**이면($K = 2, p_1 = p_2 = 0.5$): $G = 1 - 0.25 - 0.25 = 0.5$ — 이진 분류의 최댓값

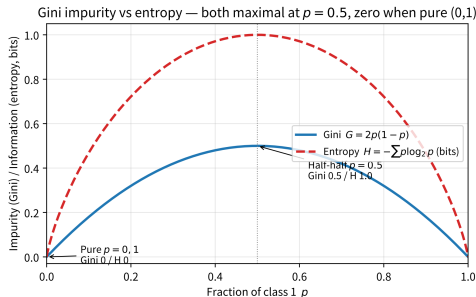
지니불순도의 두 가지 해석

(1) “잘못 고를 확률” 해석

- 노드에서 항목을 **무작위 2개 뽑아 다른 클래스일 확률**이 정확히 G
- 이진: $1 - p^2 - (1 - p)^2 = 2p(1 - p)$
- 순수($p = 1$) → “다를 확률 0”, 반반($p = 0.5$) → 최대 0.5
- 즉 $G =$ “이 노드의 예측이 **틀릴 확률**”

(2) “스무고개 정보” 해석

- 질문이 “클래스1인가?”일 때, 답이 **반반으로 갈릴수록** 정보가 가장 크다
- $G = 2p(1 - p)$ 는 $p = 0.5$ 에서 최대인 포물선



둘 다 $p = 0.5$ 에서 최대, 순수에서 0. 지니는 매끄러운 포물선, 엔트로피는 반반 근처가 더 완만.

정보이득: 나누기 전 엔트로피 – 가중평균

엔트로피(entropy, Ch. 2.3 새넌 정보이론):

$$H = - \sum_{k=1}^K p_k \log_2 p_k$$

지니불순도와 마찬가지로 순수할수록 0, 섞일수록 크다.

노드를 왼쪽(L)과 오른쪽(R)으로 나눴을 때 **정보이득**(Information Gain):

$$IG = H(\text{parent}) - \left(\frac{|L|}{|L|+|R|} H(L) + \frac{|R|}{|L|+|R|} H(R) \right)$$

- IG가 클수록 그 질문이 데이터를 잘 나눴다는 뜻
- 트리를 만들 때 각 단계에서 **IG(또는 지니 감소량)가 가장 큰 질문**을 고른다 — 스무고개의 “가장 잘 좁히는 질문을 먼저”이 알고리즘이 된 것

두 가지 “왜”: 크기 가중과 $\log_2$

왜 자식의 불순도를 “크기로 가중”하는가

- 자식 중 하나가 1개, 다른 하나가 99개 샘플일 때, 1개짜리 순수 노드의 “ $G = 0$ ”을 무겁게 세면 **불공정**
- 그래서 각 자식의 불순도를 **샘플 수 비율**로 곱해 평균한다 — “가중평균”의 이유 전부

엔트로피의 밑은 왜 $\log_2$ 인가

- 단위가 **비트(bit)**: “이 항목의 클래스를 알려면 예/아니오 질문을 **평균 몇 개** 해야 하는가”의 답
- 반반($p = 0.5$) $\rightarrow$ 1비트, 한쪽으로 쏠려 있으면 ≈ 0 비트
- 다른 밑을 써도 단위만 바뀌고 **분기 비교의 순서는 변하지 않음**

세 함수를 손으로 구현

```
def gini(labels):
    n = len(labels)
    if n == 0:
        return 0
    counts = {}
    for l in labels:
        counts[l] = counts.get(l, 0) + 1
    return 1 - sum((c / n) ** 2 for c in counts.values())

def entropy(labels):
    n = len(labels)
    if n == 0:
        return 0
    counts = {}
    for l in labels:
        counts[l] = counts.get(l, 0) + 1
    return -sum((c / n) * math.log2(c / n) for c in counts.values())

def information_gain(parent_labels, left_labels, right_labels):
    n = len(parent_labels)
    weighted_child = (len(left_labels) / n) * entropy(left_labels) + \
        (len(right_labels) / n) * entropy(right_labels)
    return entropy(parent_labels) - weighted_child
```

손으로 한 번: 두 분기 후보 비교

샘플 6개: $x = [1, 2, 3, 4, 5, 6]$, 라벨 $[A, A, A, B, B, B]$. 부모 노드는 반반이므로 $G_{\text{parent}} = 1 - 0.5^2 - 0.5^2 = 0.5$.

분기 후보	왼쪽 / 오른쪽	가중평균 불순도	Gain
$x \leq 3$	3A / 3B (모두 순수, $G = 0$)	0	0.5
$x \leq 1$	$[A] / [A, A, B, B, B]$ ($G_R = 1 - 0.4^2 - 0.6^2 = 0.48$)	≈ 0.4	0.1

- 같은 데이터인데, 어디를 자르느냐에 따라 정보이득이 5배(0.5 vs 0.1) 차이난다 — 후보 1이 훨씬 “좋은 질문”
- 엔트로피 기준도 같은 결론: 부모 $H = 1$ (비트), 후보 1: 1.0, 후보 2: ≈ 0.191

코드로 확인: 손계산과 정확히 일치

```
X = [1, 2, 3, 4, 5, 6]
y = ["A", "A", "A", "B", "B", "B"]
print("parent Gini=%.3f Entropy=%.3f" % (gini(y), entropy(y)))
for t in (1, 3):
    L = [y[i] for i in range(6) if X[i] <= t]
    R = [y[i] for i in range(6) if X[i] > t]
    wG = (len(L)/6)*gini(L) + (len(R)/6)*gini(R)
    wH = (len(L)/6)*entropy(L) + (len(R)/6)*entropy(R)
    print(f"t={t}: Gini gain={gini(y)-wG:.3f} | Ent gain={entropy(y)-wH:.3f}")
```

parent Gini=0.500 Entropy=1.000 t=1: Gini gain=0.100 | Ent gain=0.191

t=3: Gini gain=0.500 | Ent gain=1.000 — 손계산과 **정확히** 일치.

- best_split은 **모든 threshold를 하나씩** 시도해 최대 IG를 고른다 — 모든 특징에 걸쳐 반복하는 것이 DecisionTreeClassifier.fit 각 노드에서 일어나는 일
- 연속값 특징: 인접 두 값의 **중간만** 후보면 충분 $\rightarrow n$ 개 고유값 특징당 후보 $n - 1$ 개

실습: sklearn으로 트리를 배우고 “읽기”

유방암 진단 데이터(양성/악성, 30개 특징), train/test 7:3, **깊이 3**.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=0)

tree = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_train, y_train)
print(export_text(tree, feature_names=list(data.feature_names), max_depth=2))
```

|— worst concave points <= 0.14 |— worst concave points > 0.14 ...

train acc: 0.967 test acc: 0.947

- export_text가 **알고리즘이 스스로 골라낸 질문**을 그대로 보여준다
- 루트가 worst concave points인 것 = **정보이득이 가장 큰 질문**을 자동 탐색해 루트에 놓은 것
- 이 트리 하나는 사람이 **그대로 읽을 수 있다**: “0.14 이하이면(악성 쪽) worst area로 한 번 더 나누고...”

예측 경로 추적 + 특징 중요도

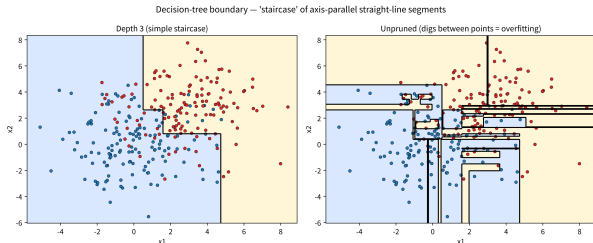
```
X_one = X_test[:1]
path = tree.decision_path(X_one)
print("경로를 지난노드 :", path.indices)      # 루트부터리프까지노드번호

imp = tree.feature_importances_
idx = np.argsort(imp)[::-1][:3]
for i in idx:
    print(data.feature_names[i], round(imp[i], 3))
```

worst concave points 0.817 worst area 0.098 area error 0.054

- `path.indices` = “이 환자가 악성으로 예측된 **이유**” — 7.3절 SHAP은 **수백 개의** 트리 경로를 합성해야 하지만, **트리 하나**의 설명은 이 한 줄이면 끝
- `feature_importances_` = 각 특징이 분기에 쓰일 때마다 **줄인 지니불순도를 누적**한 값(루트 질문과 일치)
- 단, “이 트리 전체”의 설명이지 “한 환자의 예측” 설명은 아님 (SHAP이 그 격차를 메운다)

결정 경계를 눈으로: “계단”이 생기는 이유



왼쪽: 깊이 3, 오른쪽: 완전(제한 없음).

- 2개 특징 400개 샘플(두 덩어리, 오버랩) — **둘 다 계단**
- 질문이 “ $x_j > t$?”뿐이라 경계는 **필연적으로** 축에 평행한 직선 조각 — 대각선 한 줄도 못 그린다
- 왼쪽(깊이 3): 리프 8개의 비교적 단순한 계단
- 오른쪽(완전): 학습 데이터를 100% 맞추려 **학습 점 사이로 파고드는** 복잡한 계단(리프 59개)
- 숫자로: 깊이 3 → train 0.846 / test 0.792 완전 → train **1.000** / test **0.733**

“학습 데이터를 더 잘 맞히는 모델이 더 좋은 모델”이라는 직관이 여기서 처음 깨진다.

언제 나누기를 멈추는가: 정지 조건

- 끝까지 키우면 모든 리프가 순수해진다 — 학습 데이터는 100% 맞지만 새 데이터에 취약한 **과적합** 트리
- **정지 조건**(stopping criteria) 세 가지:
 - 1 max_depth — 트리의 **높이** 상한
 - 2 min_samples_leaf — **리프에 들어갈 최소 샘플 수** (2개뿐인 리프는 “무엇이든 1:1 예측”하는 노이즈 외우기)
 - 3 min_samples_split — **분기할 수 있는 최소 샘플 수**
- 또는 트리를 다 키운 뒤 불필요한 가지를 자르는 **가지치기** (pruning)

트리에서의 “정규화”

셋 다 “더는 나누지 마”라는 같은 신호 — Ch. 6의 정규화강도 λ 에 대응한다. **트리에서 정규화란 트리를 작게(간단하게) 유지하는 것이다.**

실습: 깊이를 늘리면 — U자 곡선

```
depths = [1, 2, 3, 4, 5, 6, 8, 12, None]
for d in depths:
    m = DecisionTreeClassifier(max_depth=d, random_state=0) \
        .fit(X_train, y_train)
    print(f"max_depth={str(d):>4}: train={m.score(X_train, y_train):.3f} "
          f"test={m.score(X_test, y_test):.3f} leaves={m.get_n_leaves()}")
```

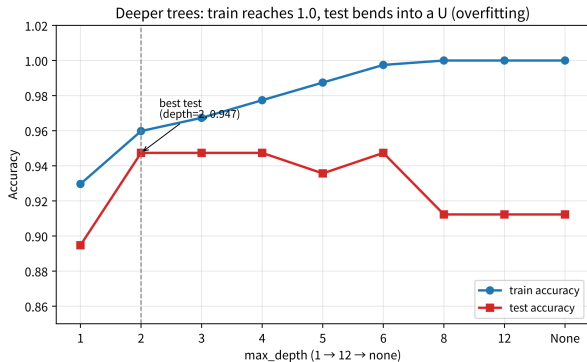
max_depth=1: train=0.930 test=0.895 leaves=2 max_depth=2: train=0.960 test=0.947 leaves=4

max_depth=3: train=0.967 test=0.947 leaves=7 max_depth=8: train=1.000 test=0.912 leaves=19

max_depth=None: train=1.000 test=0.912 leaves=19

max_depth=4는 test 0.947, =5는 0.936, =6는 0.947, =12도 test 0.912.

U자 곡선: 편향-분산 트레이드오프



train은 1.0까지 단조 증가, test는 depth 2~6에서 정점 후 depth 8에서 0.912로 떨어져 U자.

- **왼쪽**(깊이 1~2, 너무 얇은): train·test 모두 낮음 — **편향(bias)** 영역. 질문 1개(스텝)로는 부족
- **가운데**(깊이 2~6, 적당히): test **0.947** 정점, 그 부근에서 평탄하게 머문다 — **최적 깊이**
- **오른쪽**(깊이 8~None, 너무 깊은): train **1.000**(100% 외움) vs test **0.912** — **분산(variance)** 영역

실무 규칙

최적 깊이는 **test 정확도의 정점**(depth 2~6, 0.947). “train이 1.0인 가장 깊은 트리”를 고르면 U자 **오른쪽 끝** (0.912)에 떨어져 **3.5%p 나빠진다** — 검증세(또는 72의 OOB)으로 이 U자를

자주 하는 실수 1: max_depth를 제한 없이 두기

	max_depth=None	max_depth=3
train	1.000	0.967
test	0.912	0.947

- 깊이 제한이 없는 트리는 학습 데이터를 **완벽히**(100%) 외웠지만, 새 데이터에서는 오히려 **더 나쁨**(91.2% vs 94.7%)
- “학습 데이터를 더 정확히 맞추는 트리가 더 좋은 트리”라는 직관이 여기서 **완전히** 틀린다 — Ch. 4.1 편향-분산 트레이드오프가 트리에서 그대로 재현
- 깊이 제한이 없으면 분산이 극단적으로 커지고(노이즈 하나까지 다 기억), 적당한 제한은 약간의 편향을 감수해 **분산을 크게 줄임**
- **정지 조건은 선택 사항이 아니라, 반드시 검증셋으로 튜닝해야 하는 필수 하이퍼파라미터**

자주 하는 실수 2: “ $IG > 0$ ”이면 무조건 나누기

- 함정: 리프에 5개 샘플(4A, 1B)이 있는데 “그 1개 B를 떼어내자” $\rightarrow$ IG는 양수($0.32 \rightarrow 0$)
- 하지만 **샘플 1개짜리 리프**를 만든 것은 그 1개 샘플의 **라벨링 노이즈를 그대로 외운 것**
- 새 데이터에서 그 위치의 “진짜” 클래스가 A라면 틀린다

- 막는 것: `min_samples_leaf`(리프 최소 샘플 수), `min_samples_split`(분기 최소 샘플 수)
- `max_depth`는 트리의 **높이**를, `min_samples_*`는 **리프의 크기**를 막는다
- `max_depth`만으로도 상당한 효과; **클래스 불균형** (99:1)이나 **고차원**(특징 수백 개)이면 `min_samples_leaf`까지 함께

수정된 규칙

“IG가 양수이면 나뉘도 된다”가 아니라, “IG가 **충분히** 양수 **이고** 나눈 리프가 **충분히 크고**”일 때만 나누는 것 — CART 탐욕스러운 분기의 현실적 한계.

자주 묻는 질문

- **지니불순도 vs 엔트로피?** 실전에서 거의 같은 분기를 고른다. 지니는 **로그 계산이 없어 더 빠르다** → scikit-learn 등 기본값. (반반 근처에서 엔트로피는 더 완만해, 가끔 분기가 달라질 수도.)
- **왜 스케일링이 필요 없나?** kNN/SVM은 거리를 쓰니 스케일에 민감했지만, 결정 트리는 “ t 보다 큰가?”라는 **순서(비교) 정보**만 쓴다 — 2배로 늘려도, 미터를 센티미터로 바꿔도 순서 관계가 안 변한다.
- **연속 특징도 예/아니오로?** 그렇다 — 인접 고유값 **중간**만 후보 threshold(n 개 고유값 → $n - 1$ 개). 실습의 worst concave points ≤ 0.14 가 자동 골라진 cutoff.
- **회귀에도?** CART = Classification and **Regression** Trees — 리프 = 라벨 **평균**, 분기 기준 = **제곱오차 감소**(SSE 감소). 7.3절 GBDT에서 재등장.
- **탐욕스러운 분기 = 전역 최적? 아님** — 매 단계 “그 순간 IG 최대”일 뿐. 전역 최적은 **지수 시간**에 탐색 불가 → “충분히 좋고, 빠르다”는 탐욕 근사. 이 한계가 **랜덤 포레스트(7.2)와 GBDT(7.3)**로 넘어가는 동기.

7.2 트리 하나에서 숲으로: 랜덤 포레스트

트리 하나에서 숲(랜덤 포레스트)으로

- 트리 하나는 데이터를 **완벽히 외워버리기**(과적합) 쉽다
- **랜덤 포레스트**(Random Forest, Breiman 2001)의 아이디어: 서로 조금씩 다른 트리를 **여러 개** 만들고, **다수결**로 예측한다

핵심 직관

“**완전히 같은 실수를 하는 전문가 100명**”보다 “**서로 다른 실수를 하는 전문가 100명**”의 평균이 훨씬 안정적이다. 두 가지 무작위성이 트리들을 서로 덜 닮게 만들어, 개별 트리의 과적합 성향이 다수결로 **상쇄**된다.

두 가지 무작위성

1. 배깅(Bootstrap Aggregating)

- 각 트리는 원본에서 **복원추출**(중복 허용)로 **같은 크기**의 데이터를 뽑아 학습
- → 트리마다 **조금씩 다른 데이터**를 봄

2. 특징 무작위화

- 각 분기(split)에서 **전체 특징이 아니라** 무작위로 고른 **일부 특징** 중에서만 최선의 질문을 찾음
- → 같은 데이터를 봐도 **다른 질문**을 탐색

이 둘이 “트리들이 서로 닮지 않게” 만드는 방식: 하나는 **데이터**를, 다른 하나는 **각 분기에서 고려하는 특징**을 무작위화한다.

역사 1: 골튼의 소 체중 맞추기 (1889)

- 1889년, 영국 통계학자 프랜시스 골튼(Francis Galton)은 시골 박람회의 “**시상 소**” **체중 맞추기** 이벤트 결과를 조사
- 787명이 쓴 추정치의 **평균**(1,251파운드)은 실제 (1,198파운드)과 불과 **1% 이내** — 개별 추정치 중 거의 어느 것보다 정확

군중의 지혜(wisdom of crowds)

“많은 사람 각각은 틀리지만 **그 평균은** 놀라울 만큼 정확하다.” 골튼의 “평균이 개인을 이긴다”는 발견은, 개별 트리의 흔들림(분산)을 **다수결로 없애겠다는 배경**의 목표와 정확히 같은 말.

역사 2: 부트스트랩(1993) → 배깅 → Breiman(2001)

- 골튼의 원리는 약 100년 뒤, **부트스트랩**(bootstrap, Efron & Tibshirani 1993)이라는 도구로 돌아옴: **복원추출**로 원본과 같은 크기의 샘플을 계속 뽑으면 그 통계량 분포가 **진짜 분포를 근사**
- 랜덤 포레스트는 이 부트스트랩을 **트리의 학습 데이터**에 그대로 적용 — 각 트리가 “다르게 보게 만드는 것” = **배깅**(Bühlmann & Hüsler 1997)
- 마지막 조각, **특징 무작위화**는 2001년 Breiman의 “Random Forests”에서 완성: **배깅만으론**(특정 강한 특징이 있으면) 트리들이 너무 닮는다는 문제를, “각 분기에서 **특징의 무작위 하위 집합**만 고려하자”는 두 번째 무작위성으로 풀었다

(부트스트랩 + 특징 무작위화)의 **조합**이 바로 우리가 쓰는 랜덤 포레스트다.

실습: 트리가 어떤 질문을 던지나 (복습)

유방암 데이터, 깊이 3 — 7.1절과 같은 코드:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=0)

tree = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_train, y_train)
print(export_text(tree, feature_names=list(data.feature_names), max_depth=2))
print("트리 정확도: ", round(tree.score(X_test, y_test), 3))
```

루트가 worst concave points(가장 오목한 부분의 정도)로 먼저 나누는 것이 보임 — 정보이득이 가장 큰 질문을 알고리즘이 스스로 찾아낸 것.

- 대출 심사 예시처럼, 이 **트리 하나**는 “왜 이렇게 예측했는가”를 사람이 **그대로 읽을 수 있다**

손으로 한 번: “여러 전문가의 평균”이 더 안정적이는가

같은 유방암 학습 데이터에서 **부트스트랩 샘플 5개**로 깊이 제한 없는 트리를 **독립적으로** 5개 학습 (개별 트리 흔들림을 선명하게 보기 위해):

	tree 0	tree 1	tree 2	tree 3	tree 4
test acc	0.871	0.918	0.895	0.889	0.883

- 5개는 **87.1% ~ 91.8%**로 흩어짐 (평균 89.1%, **표준편차 1.55%p**) — 어떤 부트스트랩 샘플을 뽑았느냐에 따라 꽤 요동침
- 이 5개를 **100개 트리의 다수결**로 합친 랜덤 포레스트:

random forest (100 trees): **0.959**

개별 트리의 **최댓값(91.8%)보다도** 높다. 포레스트를 5번 다른 시드로 다시 학습해도 95.9%~97.7% (표준편차 **0.68%p** — 개별 트리 1.55%p보다 훨씬 좁음). **정확도 상승 + 표준편차 감소** = 서두 주장의 실측 증거.

왜? — 앙상블 분산 공식

Ch. 1.1의 **편향-분산 분해**: 예측 오차 = **편향**(본질적으로 못 맞히는 부분) + **분산**(데이터가 조금만 바뀌어도 예측이 흔들리는 부분). 깊이 제한이 없는 결정 트리는 **분산이 매우 큼** (7.1: max_depth=None 91.2% < 깊이 3의 94.7%).

같은 설계의 트리를 T 개, 각각 다른 부트스트랩 샘플로 학습. 각 트리의 test 예측 오차 표준편차 σ , 두 트리 간 상관계수 ρ 라면 T 개 **평균(다수결 앙상블)의 오차 분산**:

$$\text{Var}[\text{앙상블}] = \frac{\sigma^2}{T} (1 + \rho(T - 1))$$

(개별 예측 **평균**의 분산 공식: $T\sigma^2 + T(T - 1)$ 개의 공분산 $\rho\sigma^2$, T^2 로 나누면 됨.)

공식을 극한으로 — 그리고 ρ 를 손으로

두 극한

- **완전히 독립** ($\rho = 0$): 분산 σ^2/T , 표준편차 $\sigma/\sqrt{T}$ 로 **줄어든다**
- **완전히 같으면** ($\rho = 1$): 분산 σ^2 **그대로** — “완전히 같은 실수를 하는 전문가 100명”은 전문가 1명과 **같다**

현실은 둘 사이 ($0 < \rho < 1$): **무작위성이 클수록** ρ 가 작아져 $\sigma/\sqrt{T}$ 에 가까워진다.

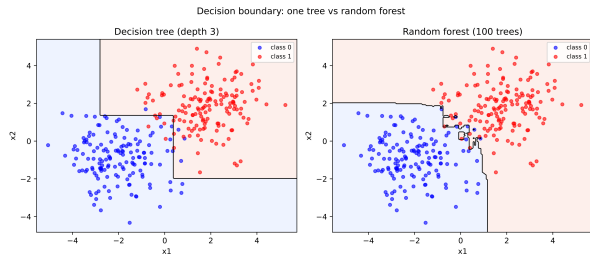
손으로

- 개별 트리 $\sigma \approx 1.55\%p$, 독립이라면 100개 평균은 $\frac{1.55}{\sqrt{100}} = 0.155\%p$
- 실측: 5번 다른 시드 포레스트가 **0.68%p**로 모였다 — “완전히 독립(0.155%p)은 아니지만 개별 트리보다 훨씬 안정”
- 0.68%p를 공식에 대입하면 $\rho \approx 0.2$ — 같은 강한 특징을 다 보지만 **상관이 낮은 쪽**

결론

두 가지 무작위성의 역할 = 정확히 이 ρ 를 줄이는 것. ρ 가 1에 가깝지 않다는 것이 “앙상블이 단일 트리보다 좋은 이유”의 **전부**.

결정 경계를 눈으로: “계단”에서 “곡선”으로



왼쪽: 트리 하나(깊이 3), 오른쪽: 랜덤 포레스트(100개). 2개 특징 300개 샘플.

- 왼쪽: **축에 평행한 직선 조각이 이어진 계단형** — “ $x_1 > t$?” 같은 축에 평행한 질문만 하므로 필연적
- 오른쪽: 100개 트리의 **다수결** → 각 트리의 계단이 조금씩 달라 **훨씬 부드러운 곡선**
- 이 “부드러짐” = 앞 절 **분산 감소의 시각화**: 단일 트리의 날카로운 계단은 잡음에도 쉽게 모양이 변하지만 (높은 분산), 100개 계단의 평균은 잡음이 **상쇄**되어 안정적

“트리가 하나면 계단, 숲이면 곡선”이라는 앙상블의 본질을 2D 데이터에서 눈으로 직접 보는 장면.

OOB(out-of-bag): 검증 세트 없이도 일반화 성능을

- 각 트리는 **복원추출**된 부트스트랩 샘플로 학습하는데, 복원추출은 원본 샘플 중 **약 37%**를 “**한 번도 안 고른다**”
- 그 37% = **OOB**(out-of-bag, bag 밖) 샘플 — 해당 트리가 **보지 못한** 샘플
- 이 트리가 “보지 못한” 샘플을 예측하게 하면, 학습을 마치자마자 **새 데이터에 대한 예측**을 할 수 있다 — **별도 검증 세트를 떼어놓을 필요가 없음**

실무적 중요성

데이터가 적어 검증 세트를 따로 떼기 아까울 때에도 “이 모델이 과적합이 아닌가”를 **학습 중 즉시** 확인할 수 있다.

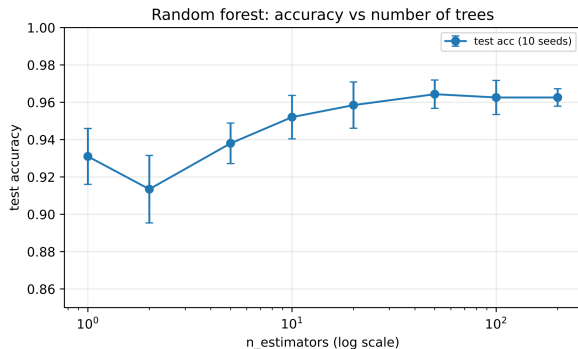
OOB 실습: test와 거의 붙는다

```
rf = RandomForestClassifier(n_estimators=100, oob_score=True, random_state=0)
rf.fit(X_train, y_train)
print("OOB score:", round(rf.oob_score_, 3))
print("test score:", round(rf.score(X_test, y_test), 3))
```

OOB score: 0.955 test score: 0.959

- OOB 점수(0.955)가 진짜 test(0.959)와 **거의 붙는다**
- **주의:** oob_score_는 oob_score=True로 학습했을 때만 **속성**으로 생기며, oob_score()라는 메서드가 아니다

트리 개수를 늘리면: 정확도는 오르고 흔들림은 줄어든다



test 정확도(선)와 표준편차(오차막대)를 $n_{\text{estimators}}$ (로그 스케일)의 함수로.

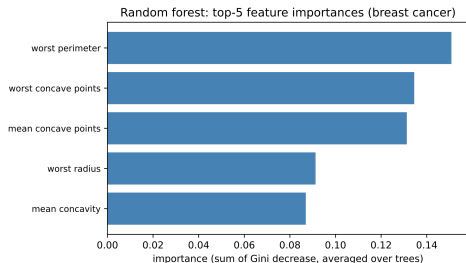
$n_{\text{estimators}}$ 1→200까지, 각 시드 10개 평균:

- 1 정확도는 1→10개 정도로 **빠르게 올랐다가** 거의 **수평**(수확체감)
- 2 표준편차(오차막대)는 트리가 늘수록 **줄어든다** — “전문가”가 많을수록 소수의견에 흔들리지 않음
- 3 정확도가 **떨어지는 지점이 없다** — 트리를 더 늘려도 **과적합이 안 온다**(7.3 GBDT와 대조) → 실전은 정체 지점보다 여유를 두고 **100~300**이 안전

숲 전체가 어떤 특징을 보는가: 중요도

- 단일 트리의 `feature_importances_` = “그 트리가” 어떤 특징을 많이 봤는지
- 100개 트리의 그것 = **각 트리가 분기마다 줄인 지니불순도(7.1)를 특징별로 누적해 100개에 걸쳐 평균** — “숲 전체가” 어떤 특징을 보는지
- 유방암: `worst perimeter(0.151)` > `worst concave points(0.135)` > `mean concave points(0.131)`
- “worst(최악)” 특징의 상위권 = 실제 진단에서 “가장 나쁜 상태”가 가장 결정적이었음을 뜻함

단, 이 중요도는 **상관관계에 편향**될 수 있다 — 7.3의 SHAP이 이 한계를 보완하는 “예측 단위” 설명.



상위 5개 특징 중요도(지니 감소량, 트리 전체 평균).

특징 무작위화: “강한 특징”이 루트를 독점하는가

배깅만 하는 BaggingClassifier(특징 무작위화 **없음**)와 랜덤 포레스트(**있음**)를 50개 트리로 만들고, **각 트리가 루트에서 어떤 특징을 쓰는지** 세면:

```
dom = list(data.feature_names).index("worst concave points")
bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(max_depth=6, random_state=0),
    n_estimators=50, random_state=0).fit(X_train, y_train)
rf = RandomForestClassifier(n_estimators=50, max_depth=6, random_state=0) \
    .fit(X_train, y_train)

root_frac = lambda clf: np.mean([t.tree_.feature[0] == dom
                                  for t in clf.estimators_])
print("bagging-only: ", round(root_frac(bag), 2))
print("random forest: ", round(root_frac(rf), 2))
```

bagging-only: 0.40 random forest: 0.20

- 배깅만: 50개 중 **40%**가 루트에서 worst concave points 사용 → RF는 **절반(20%)**
- 이유: RF는 분기마다 30개 중 무작위 일부(기본 $\max_features = \sqrt{30} \approx 5$)만 후보로 보므로 강한 특징이 “이번엔 후보에 안 들어가는” 경우가 생김
- 결과: 트리들이 **서로 다른 질문**을 탐색 → ρ 를 낮추어 분산 감소를 **더 크게**

자주 하는 실수: `n_estimators`를 너무 적게

- 배깅의 효과는 트리 개수가 늘수록 (수확체감이 있긴 하지만) **꾸준히 안정** — → `n_estimators=3`처럼 적게 쓰면 앞 실습의 “개별 트리 5개 흠어짐”과 크게 다르지 않은 **불안정** 결과 (“전문가”가 너무 적으면 소수 의견에 쉽게 흔들림)
- 실전: `n_estimators=100` 이상을 **기본**으로 시작, 검증 성능이 “정체 구간”이 되는 지점을 확인

GBDT와의 대비

트리를 더 추가하는 것은 (**GBDT와 달리**) 과적합을 일으키지 않으므로, “너무 많이 쓰는” 걱정은 **훨씬 적어도 됨** (다만 학습·예측 **속도**만 느려진다).

자주 묻는 질문

- **두 무작위성 모두 필요한가?** 배경만으론, **특정 강한 특징**이 있으면 샘플이 달라져도 거의 모든 트리가 그 특징으로 루트를 나눠 **너무 얕아버림** → 특징 무작위화까지 하면 어떤 트리는 그 특징을 아예 후보에서 못 봄 (앞의 40%→20%).
- **max_features**는? **분류** 기본값 $\sqrt{n}$, **회귀** 기본값 $n/3$. 특징 1000개면 $\sqrt{} \rightarrow 31$ 개. 기본값은 **출발점**일 뿐 — 검증셋으로 $\sqrt{}$, $\log_2$, 1.0(전체)를 비교. 특징이 독립적 → 더 많이(값 크게), 상관관계 강하면 → 덜.
- **predict_proba**는? 각 트리의 **클래스 확률 평균** — 다수결(predict)이 “클래스를 더 많이 말았는가”를 세는 반면 부드러운 **확률**을 줌. OOB 계산·임계값 튜닝에 쓰임.
- **사람이 “이 예측이 왜?”, 읽을 수 있나? 아니요** — 100개 트리는 개별 예측 하나에 어떤 트리의 어떤 특징이 기여했는지 눈으로 따지기 어렵다. **7.3의 SHAP**이 정확히 이 문제를 푼다.
- **feature_importances_**? 같은 방식(지니 감소 누적)이나, RF는 **독립적** 학습 / GBDT는 **순차적**(잔차) 학습이라 같은 데이터에도 **중요도 순위가 달라질 수 있다**.

7.3 GBDT와 SHAP

다른 조직 방식: GBDT

7.2의 랜덤 포레스트 = “서로 독립적인 전문가 100명의 다수결”. **GBDT**(Gradient Boosted Decision Trees, Friedman 2001)는 전혀 다른 조직 방식:

“선임이 실수를 남기면, 다음 직원이 바로 그 실수를 고치는” 순환 근무

- 랜덤 포레스트: 트리를 **독립적으로** 만들고 **다수결**
- GBDT: 트리를 **하나씩 순차적으로** 추가하고, **매번** 새 트리는 “지금까지의 예측이 아직 못 맞힌 부분 (**잔차**, residual)”을 목표로 학습

GBDT: 순차적으로 잔차를 추적

- 목표: T 개 트리 $f_1, \dots, f_T$ 를 **순차적으로 더해** 예측 $F_T(x) = \sum_{t=1}^T f_t(x)$
- t 번째 트리 학습 시: 지금까지 쌓은 예측 F_{t-1} 의 **잔차**

$$r^{(i)} = y^{(i)} - F_{t-1}(x^{(i)})$$

을 **새 목표값**으로 삼아 그 잔차를 예측하는 트리를 학습

- **learning_rate**(축소율, shrinkage): 각 트리의 기여를 일부러 **줄여서** 한 트리가 과적합하는 것을 막고, 여러 트리가 조금씩 나눠서 학습하게 함 — Ch. 2 경사하강법의 **학습률**과 같은 역할
- “잔차를 향해 조금씩 이동” = **함수 공간에서의 경사하강법** → 그래서 이름이 **gradient boosting**
- **XGBoost**(2016), **LightGBM**(2017) = 실무 최적화 라이브러리, 지금도 **정형(tabular) 데이터 대회(Kaggle)**에서 가장 자주 우승하는 축

핵심 루프: 잔차를 목표로 트리 하나씩

```
def gbd_t_fit(X, y, n_trees, learning_rate):
    trees = []
    predictions = [0.0] * len(y)  #  $F_0(x) = 0$ 
    for t in range(n_trees):
        residuals = [y[i] - predictions[i] for i in range(len(y))]
        tree = fit_single_tree(X, residuals)  # 잔차를 목표로 트리 하나
        trees.append(tree)
        for i in range(len(y)):
            predictions[i] += learning_rate * tree.predict(X[i])
    return trees
```

- 실무 라이브러리의 스텝 단위는 보통 **깊이 3~6짜리 작은 트리**(아래 1차원 실습은 스텝/깊이 1) — 핵심은 “각 스텝이 **잔차를 목표로 작은 함수 하나**를 추가한다”는 구조, 스텝의 크기는 하이퍼파라미터
- F_{t-1} 이 어느 정도 맞고 있다면 남은 잔차(오차)는 원래 y 보다 **작다** — 그 작은 오차를 새 트리가 다시 줄이면 $F_t = F_{t-1} + \eta f_t$ 의 오차는 **한 단계 더 작아짐**
- 계속 추가하면 학습 데이터 오차는 이론상 계속 줄지만, 실전은 **검증 성능이 나빠지기 시작하는 시점**(과적합)에서 **early stopping**(Ch. 6 검증 데이터 원칙과 동일)

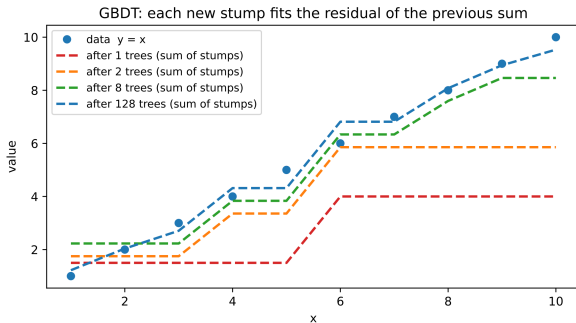
잔차 추적을 눈으로: 스텝 8개로 직선 근사

1차원 데이터: 목표 $y = x$ ($x = 1, \dots, 10$). 쓸 수 있는 모델은 **스텝**(깊이 1: threshold보다 작으면 a , 크면 b)뿐 — 스텝 하나로써는 직선을 절대 못 그린다. learning_rate=0.5로 **8단계** 추적:

스텝	threshold / left / right	train MSE
1	5 +3.000 / +8.000	11.125
2	3 +0.500 / +3.714	3.826
3	7 +0.617 / +3.143	1.403
4	8 +0.342 / +2.071	0.690
5	2 -0.729 / +0.612	0.385
6	9 +0.054 / +1.230	0.270
7	1 -0.892 / +0.195	0.185
8	1 -0.446 / +0.097	0.163

- train MSE: 스텝 0개(모두 0 예측)의 **38.5**에서 8단계 만에 **0.16**까지
- 1단계: 왼쪽 절반 $+3.0 \times 0.5$, 오른쪽 $+8.0 \times 0.5$ 로 상수 $\rightarrow$ 2단계 함수는 **기울기를 못 그려** 남은 오차가 “기울기”로 $\rightarrow$ 2단계는 threshold=3을 **1단계의 왼쪽 구간 내부**에 더해 그 기울기를 수정
- tree 7, 8은 이전 트리가 이미 쓴 threshold(=1)를 **다시** 골라 진폭만 미세 조정($-0.892 \rightarrow -0.446$) — “새 트리가 항상 새 위치”가 아니라 **“남은 잔차를 가장 잘 분기하는 곳”**

각 스텝은 “앞 단계의 잔차만” 배운다



$y = x$ 를 스텝 1·2·4·8개의 합으로 근사 — 각 단계의 계단형 합이 직선에 가까워진다.

- 스텝(2단계 함수)은 **표현력이 거의 없는 기초 블럭** — 그 잔차를 **순서대로** 추격하는 것만으로 직선(심지어 곡선)을 자유롭게 근사
- 랜덤 포레스트 = 여러 트리의 **다수결**
- GBDT = 여러 트리의 **연속 수정**(correction)
- **같은 트리 합**이지만, 그 합이 만들어지는 **방식이 정반대**

왜 “Gradient”인가: 잔차 = (음의) 기울기

회귀에서 손실을 **제곱오차** $L(F) = \frac{1}{2} \sum_i (y^{(i)} - F(x^{(i)}))^2$ 로 두면, L 을 $F(x^{(i)})$ 로 미분하면:

$$\frac{\partial L}{\partial F(x^{(i)})} = F(x^{(i)}) - y^{(i)} = -r^{(i)}$$

기울기는 정확히 잔차의 음수(제곱손실 기준).

- GBDT 스텝을 다시 쓰면: $F_t(x) = F_{t-1}(x) - \eta g_t(x)$, 여기서 g_t 는 $-r^{(i)}$ (= 기울기)를 맞추는 트리
- Ch. 2 경사하강법 $\theta \leftarrow \theta - \eta \nabla_{\theta} L$ 과 **구조가 완전히 같은 식** — 차이 하나: 경사하강법은 **파라미터 공간**에서 θ , GBDT는 **함수 공간**에서 F 자체를 옮김
- **Friedman(2001)의 핵심 통찰**: “함수를 파라미터로 가지는 모델을 경사하강으로 학습한다” — 손실만 미분가능하면 어떤 손실이든 적용 가능(분류는 로지스틱 손실의 기울기 → predict_proba가 나오는 분류기)

역사: 부스터(1990) → AdaBoost(1997) → XGBoost(2016)

연도	이름	내용
1990	booster	Schapire가 “순차 추가”의 부스팅 이라는 이름 붙임
1997	AdaBoost	Freund & Schapire: 틀린 샘플의 가중치 올려 다음 모델이 그쪽에 집중 → 첫 실용 알고리즘
2001	GBDT	Friedman: “부스팅 = 함수 공간의 경사하강 ” 해석 → 명확한 이론적 지위, 어떤 손실에든 적용할 일반 공식
2016	XGBoost	Chen & Guestrin:

실습: GBDT vs 랜덤 포레스트 (같은 데이터)

7.2절의 유방암 데이터:

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=0).fit(X_train, y_train)
gbdt = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,
                                  random_state=0).fit(X_train, y_train)

print("RandomForest:", round(rf.score(X_test, y_test), 3))
print("GBDT:           ", round(gbdt.score(X_test, y_test), 3))
```

RandomForest: 0.959 GBDT: 0.977 상위 3 중요도: worst concave points 0.445, mean concave points 0.275, worst perimeter 0.071

- GBDT가 RF보다 **근소하게 더 높음**(0.977 vs 0.959) — “독립적으로 합치는 것”보다 “**잔차를 순차적으로 쫓는 것**”이 조금 더 정교하게 맞춤
- feature_importances_는 **모델 전체**의 설명이지 “이 환자 한 명의 예측” 설명은 아님 → 개별 예측 하나를 설명하려면 **SHAP**이 필요

중요도 순위가 바뀐다: 학습 과정의 반영

	랜덤 포레스트 (7.2)	GBDT (이번 실습)
1위	worst perimeter (0.151)	worst concave points (0.445)
2위	worst concave points (0.135)	mean concave points (0.275)
3위	mean concave points (0.131)	worst perimeter (0.071)

- **같은 데이터, 같은 지표**(지니 감소량)인데 **순위가 바뀐** — 트리를 만드는 방식이 다르기 때문
- RF의 각 트리는 데이터 전체의 분포를 “그대로” 봄; GBDT의 각 트리는 **이전 트리의 잔차**만 봄 — 잔차에는 원래 데이터의 구조가 아니라 **“아직 못 맞춘 오차의 구조”**가 남아 있음
- → feature_importances_는 **“모델의 학습 과정을 반영한 값”**이지 **“데이터의 속성”**이 아니다
- 이 제한이 **“예측 단위”의 SHAP**으로 넘어가야 하는 두 번째 이유

GBDT만이 치르는 가격: overfitting 곡선

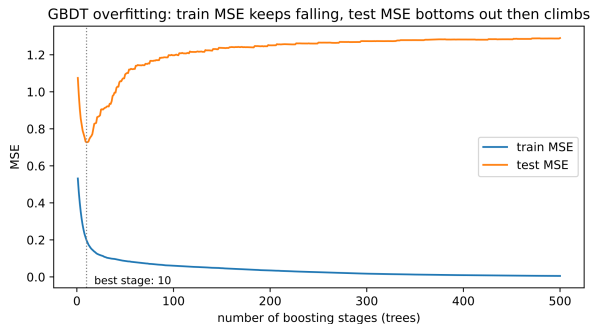
소음 섞인 1차원 함수 ($y = \sin 1.2x + 0.5 \cos 2x$ + 잡음, 200 학습/100 검증)을 **500단계** 학습, staged_predict로 **매 단계** train/test MSE 추적:

```
reg = GradientBoostingRegressor(n_estimators=500, learning_rate=0.1,  
                                max_depth=3, random_state=0).fit(Xtr, ytr)  
train_mse = [mean_squared_error(ytr, p) for p in reg.staged_predict(Xtr)]  
test_mse   = [mean_squared_error(yte, p) for p in reg.staged_predict(Xte)]  
best_stage = int(np.argmin(test_mse)) + 1
```

검증 MSE 최저점: 10단계 (test_MSE=0.727)

500단계까지 계속하면: train_MSE=0.005 test_MSE=1.290

과적합을 숫자로: 10단계 vs 500단계



train MSE는 계속 0.005까지, test MSE는 10단계 최저 후 1.290까지 다시 올라감.

왜 GBDT만? RF의 101번째 트리는 데이터를 **처음부터 다시** 보고 다른 트리의 노이즈 오류를 “따라잡아야 할 의무”가 없다(다수결이 희석). GBDT의 101번째 트리는 앞 100개 합이 **아직 못 맞춘 것 (= 학습 데이터의 노이즈 잔차)을 의도적으로 목표로 삼는다** — 적당히 쌓이면 잔차 대부분은 **라벨링 노이즈**인데, GBDT는 그것을 “정당한 학습 목표”로 받아들인다.

세 숫자가 전부 핵심:

- 1 **train MSE는 0.005까지 계속 하락** — 500개 트리가 학습 데이터의 잡음 하나까지 **거의 완벽히 외움**
- 2 **test MSE는 10단계에서 최저(0.727)**
- 3 **500단계까지 하면 1.290까지 다시 상승** — 최저점 대비 **약 두 배**로 악화, “더 오래 학습한 모델”이 정점의 **절반 수준**

실무 규칙: early stopping이 필수

- `n_estimators`는 RF처럼 “많을수록(안정적으로) 좋다”가 **아님** — 검증 곡선의 **정점(최저 MSE)**을 찾아 거기서 멈추는 **early stopping이 필수**(`sklearn validation_fraction` 옵션으로 내장)
- `learning_rate`를 **작게**($0.1 \rightarrow 0.01$) 하면 각 스텝이 작아져 같은 수준까지 **더 많은 단계**가 필요 → 정점의 위치·높이가 **완만해져 early stopping이 정밀해짐**

표준 워크플로

“`learning_rate`를 작게 하고, `n_estimators`를 **early stopping**으로 결정” — 실무 표준.

- 이 “정점을 지나 다시 올라가는 곡선” = Ch. 1.1 **편향-분산 트레이드오프**: 트리 적으면(불충분 학습) **편향**이 크고, 많으면(과적합) **분산**이 큼
- RF는 오른쪽(분산) 끝이 **거의 평탄**해서 “많게 해두면 된다”, GBDT는 **가파르게 올라서** “정점을 찾아 멈춰야 한다”

정확한데 설명이 안 된다는 문제 → SHAP

GBDT는 강력하지만 **수백 개의 트리가 얹혀** 있어 “이 예측 하나”에 어떤 특징이 얼마나 기여했는지 알기 어렵다.

- **SHAP**(SHapley Additive exPlanations) = **게임이론의 새플리값**(Shapley value)을 빌려, 예측 하나를 “**각 특징이 기여한 양**”으로 **정확히 분해**
- 원래 새플리값 = “여러 플레이어가 **협력 게임**에 참여할 때, 각 플레이어의 **공정한 몫**은 얼마인가” — 모든 가능한 **참여 순서**에서 그 플레이어가 합류함으로써 **늘어난 가치를 평균**
- **역사**: 1953년 로이드 새플리, “Value of a Cooperative Game” — “10명의 건설업자가 협력해 건물을 지으면 각각에 얼마를 공정하게?” 2012년 새플리, **노벨 경제학상**. 60년 뒤, 2017년 Lundberg & Lee가 이 도구를 **신경망·GBDT를 포함한 거의 모든 모델**의 예측 설명으로 가져옴:

정리

“어떤 모델이든, 예측 하나를 특징들의 기여로 분해하려면 **새플리값이 정답**.” 게임이론의 “공정한 분배”와 기계학습의 “예측 설명”이 **동일한 수식**으로 풀리는 도구.

SHAP의 핵심: 가산성(additivity)

- SHAP은 **특징들을 플레이어들로 바꿔 적용**: 특징 j 의 SHAP값 ϕ_j = 특징들이 하나씩 추가되는 **모든 가능한 순서**에서 “특징 j 가 추가됨으로써 예측값이 변한 정도”의 **평균**

핵심 성질(가산성, additivity):

$$f(x) = \phi_0 + \sum_{j=1}^n \phi_j$$

- ϕ_0 = **기준값**(baseline, 전체 데이터의 평균 예측)
- ϕ_j = 특징 j 가 기준에서 예측값을 얼마나 **밀어올렸는지**(양) 또는 **밀어내렸는지**(음)
- 모든 ϕ_j 의 합이 **정확히** 예측값 - 기준값과 같다는 것이 SHAP의 **핵심 보장**
- 예: “신용점수 기여 -0.3 , 소득 기여 $+0.1, \dots$ ”처럼 **합이 정확히 맞는 분해**

2개 특징 예제: 순서 두 가지의 한계기여 평균

특징이 2개(A, B)뿐이면 가능한 추가 순서는 **두 가지**: $A \rightarrow B$ 와 $B \rightarrow A$.

$$\phi_A = \frac{1}{2} [(f(\{A\}) - f(\{\})) + (f(\{A, B\}) - f(\{B\}))]$$

- 특징이 많아지면 순서 수는 $n!$ 로 **폭발** → 실제 SHAP은 **샘플링으로 근사**
- 장난감 모델($f(\{\}) = 10, f(\{A\}) = 16, f(\{B\}) = 13, f(\{A, B\}) = 20$)에 대입:

$$\phi_A = \frac{1}{2} [(16-10)+(20-13)] = \frac{1}{2}[6+7] = 6.5 \quad \phi_B = \frac{1}{2} [(13-10)+(20-16)] = \frac{1}{2}[3+4] = 3.5$$

검산 + 교차항

$\phi_0 + \phi_A + \phi_B = 10 + 6.5 + 3.5 = 20 = f(\{A, B\})$ — **정확히 맞음**. $\phi_A = 6.5$ 는 단순히 $f(\{A\}) - f(\{\}) = 6$ 이 **아님** — “A 혼자 합류(6)”와 “B가 이미 있는데 A 합류(7)”의 **평균**. 1의 차이는 B가 이미 기여한 부분이 A를 “흡수”한 **교차항 (synergy)**을 두 플레이어에 **공정하게** 나눈 것 — 이 분배가 새플리값을 단순 “순서별 차이”에서 **공정한 분배 규칙**으로 만들어주는 핵심.

실습: 코드로 새플리값 직접 계산

특징 2개라 정밀 계산이 가능:

```
f = {frozenset(): 10, frozenset("A"): 16,
      frozenset("B"): 13, frozenset("AB"): 20}

phi_A = 0.5 * ((f[frozenset("A")] - f[frozenset()]
               + (f[frozenset("AB")] - f[frozenset("B")]))
               + (f[frozenset("AB")] - f[frozenset("A")]))
phi_B = 0.5 * ((f[frozenset("B")] - f[frozenset()]
               + (f[frozenset("AB")] - f[frozenset("A")]))
               + (f[frozenset("AB")] - f[frozenset("B")]))

print(f"phi_A = {phi_A}, phi_B = {phi_B}")
assert 10 + phi_A + phi_B == 20
```

phi_A = 6.5, phi_B = 3.5

검산: $10 + 6.5 + 3.5 = 20.0$ ($f(\{A,B\}) = 20$)

- 특징이 **30개**가 되면 순서 $30! \approx 2.65 \times 10^{32}$ 개 — 우주 나이만큼 초박사들 동원해도 못 셈
- 실전 SHAP의 두 **근사 전략**:
 - ① **퍼머테이션 샘플링**: $n!$ 개 중 작은 수(수백~수천개)를 무작위로 골라 평균
 - ② **트리 구조를 이용한 정밀 계산**: GBDT의 분기 경로를 따라 “특징 j 합류 시 예측 변화”를 **모든 경로의 합**으로 효율 계산 → RF/GBDT에 한해 **근사 없이** 정밀 (shap 패키지 TreeExplainer)
- “경로”가 없는 신경망 등: 퍼머테이션 또는 **통합 기대(KernelSHAP)** 같은 근사

실습: 실제 GBDT에 퍼머테이션 근사 SHAP

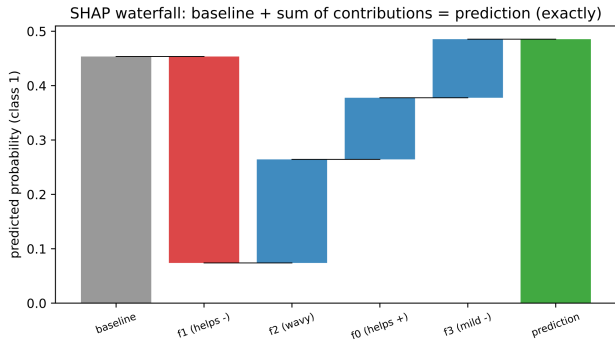
알고 만든 4개 특징 합성 데이터: 신호 $z = 0.9f_0 - 0.8f_1 + 0.5 \sin f_2 - 0.3f_3 + \text{잡음}$, 양수면 클래스 1. GBDT(80트리, depth 2) 학습 후, 예측 확률 0.3~0.7 사이인 점 하나를 골라 **400회** 퍼머테이션 근사:

	값
test accuracy	0.758
고른 점	index=113, pred_prob=0.485, true_label=0
f_0 (강한 +)	$\phi = +\mathbf{0.113}$
f_1 (강한 -)	$\phi = -\mathbf{0.380}$
f_2 (sin, 비선형)	$\phi = +0.190$
f_3 (약한 -)	$\phi = +0.108$
baseline ϕ_0	0.454
prediction	0.485
prediction - baseline	+0.032
sum of all ϕ	+0.032 (가산성 오차: 0.000000)

- **가산성이 정확히 성립:** $0.454 + (+0.113 - 0.380 + 0.190 + 0.108) = 0.485$ — 400회 근사인데도 오차 0

- **부호가 신호 방향과 대체로 일치:** f_0 약 신호 $\rightarrow +0.113$ f_1 음 신호 $\rightarrow -0.380$

워터폴트 플롯: 합이 정확히 일치하는 분해

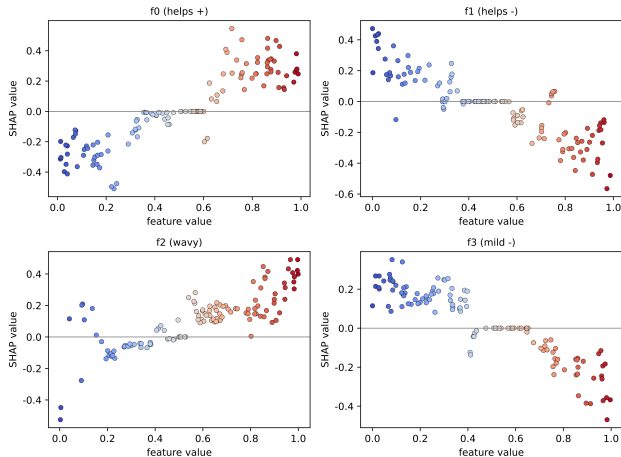


기준값(0.454)부터 기여를 크기순으로 쌓아 실제 예측값(0.485)에 **정확히** 도달.

- **워터폴트(waterfall) 플롯** = 가산성을 **시각화**: 기준값부터 각 특징의 기여를 하나씩(크기순) 쌓아가면 마지막에 **정확히** 실제 예측값에 도착
- “합이 정확히 일치한다” = SHAP이 “대략적인 설명”이 아니라 **정확한 분해**
- 은행 예: “신용점수 -0.3 , 소득 $+0.1$ ”의 합이 정확히 “승인 확률이 0.2 낮아졌다”와 일치해야 하는 것과 같음

summary 산점도: 모델 전체의 중요도 + 방향

SHAP summary: spread = importance, sign pattern = direction



- **흩어짐 = 중요도**: f_0, f_1 은 ϕ 가 크게 흩어짐 (± 0.4 이상) — “예측을 크게 움직임”, f_3 은 좁은 범위 (± 0.1 이하) — “여기선 영향 약함”
- **부호 패턴 = 방향**: f_1 은 특징값 클수록 음의 방향 (밀어내림), f_0 은 양의 방향 — 두 가지가 **동시에** 보임
- feature_importances_ (지니 감소, “특징의 **사용 빈도**”)와 달리, SHAP summary는 “**예측에 대한 실제 기여의 방향과 크기**”를 보여준다는 점에서 **더 직접적**

전체 test 120개 점의 ϕ_j 분포. x축 = 특징 값, y축 = SHAP 값, 색 = 특징 값 크기.

자주 하는 실수: GBDT의 $n_estimators$ 를 RF처럼 “크게”

RF는 n 1→200까지 늘려도 정확도가 **떨어지는 지점 없음** → “100~300으로 여유 있게” 잡으면 됐지만, **GBDT에는 이 규칙이 적용되지 않음.**

	랜덤 포레스트	GBDT
트리의 관계	독립적 (다수결)	순차적 (잔차 추격)
트리를 늘리면	정확도 ↑(수확체감), 분산 ↓	정확도 ↑ → 정점 → 정확도 ↓
$n_estimators$	“많을수록 안정” — 100~300 권장	early stopping 으로 결정 (검증 곡선 최저점)
learning_rate	해당 없음	0.01~0.1, 작을수록 정밀한 early stopping

자주 묻는 질문

- **GBDT가 왜 RF보다 과적합에 취약한가?** RF는 트리들이 독립적이라 새 트리가 이전 트리의 실수를 “따라 하지” 않음. GBDT는 **매 트리가 이전까지의 잔차를 목표로 학습** — 라벨링 오류가 섞여 있으면 뒤따르는 트리가 계속 “그 오류를 맞히려” 노력해, 트리를 너무 많이 추가하면 **노이즈까지 학습**. → learning_rate 작게 + n_estimators early stopping이 특히 중요.
- **SHAP 값이 크다고 “좋은” 기여인가? 아님** — 부호가 방향, 크기가 세기만 알려줄 뿐 “좋다/나쁘다”를 말해주지 않음. 예: 대출 심사 “신용점수 -0.3 ” = “이 신청자의 경우 승인 확률을 0.3 낮추는 방향”이지, 신용점수라는 특징이 나쁘다는 뜻이 아님 (다른 특징 소득은 $+0.5$ 로 반대 방향 가능, **합이 정확히 최종 예측이 됨** = 가산성).
- learning_rate를 $0.1 \rightarrow 0.01$ 로? 스텝 크기가 $1/10 \rightarrow$ **약 10배 더 많은 단계 필요**. 장점: (1) 정점이 **완만**해져 정확히 멈추는 단계가 극단적으로 일찍 오지 않음, (2) 한 트리의 과적합이 전체에 미치는 영향 감소. 표준 조합: $lr=0.01 \sim 0.05 + n=500 \sim 2000$ (early stopping 결정) — 학습 10배가 “정밀 early stopping”을 위한 투자.
- **SHAP값의 합이 예측값과 정확히 같은 게 우연인가? 아님** — **가산성** 성질이 보장. 각 순서에서 “빈집합 $\rightarrow$ 전체 집합”으로 이어지는 한계기여의 합은 **항상** $f(x) - \phi_0$ 와 정확히 같고(경로의 총 변화량은 **경로 순서에 무관**), 그 평균도 동일 $\rightarrow$ 400회 근사에도 **오차 0.000000**.

이 장의 한 줄: 정확도의 대가, 그리고 되찾기

한 줄

결정 트리는 “사람이 읽을 수 있는 if-then 규칙의 나무”이면서 동시에, “얼마나 잘 나누는가”를 **정확히 수식으로 짤 수 있는** 알고리즘이다.

- 트리를 **여러 개 합치면**(랜덤 포레스트, GBDT) **정확도는 올라가지만** 그 읽기 쉬움을 **잃는다**
- **SHAP**은 그 대가로 잃은 설명가능성을, **게임이론이라는 전혀 다른 도구로 되찾아오는** 시도
- **7.1** — 좋은 질문 = **지니불순도 감소량(정보이득) 최대**. U자 곡선: **test 정점**이 최적 깊이.
- **7.2** — **독립적 다수결**. 두 무작위성이 ρ 를 줄여 **분산** 감소. 계단 $\rightarrow$ 부드러운 곡선.
- **7.3** — **순차적 잔차 추격** = 함수 공간의 경사하강. 더 정밀한 대신 **early stopping**이 필수. SHAP(가산성)으로 예측 하나를 **정확히** 분해.

7.1절의 “트리 하나를 잘 나누는 원리”는 이 장의 **전부에서 재사용**되었다.

이번 주차 정리

- ① **7.1 — 질문을 숫자로:** 지니불순도($1 - \sum p_k^2$, “잘못 고를 확률”)와 정보이득으로 “좋은 질문”을 고름. 결정 경계는 **계단**. 최적 깊이 = U자 곡선의 **test 정점** (0.947), `max_depth=None`(test 0.912)은 과적합.
- ② **7.2 — 독립적 다수결:** 배경 + 특징 무작위화로 상관계수 ρ 를 줄여 분산을 감소 ($\sigma/\sqrt{T}$ 에 가까워짐). OOB로 **검증 세트 없이** 성능 평가. 트리를 늘려도 **과적합 안 옴**.
- ③ **7.3 — 순차적 잔차 추격** = 함수 공간의 경사하강 (잔차 = 음의 기울기). RF보다 정확(0.977)하지만 **early stopping 필수**(10단계 최저 → 500단계에서 2배 악화). SHAP(새플리값, 가산성)으로 잃은 설명가능성 **되찾기**.

다음 주 — Chapter 8. Block A 캡스톤

“이 문제에 어떤 모델이 맞는가”를 스스로 고르는 주차. 이번 주의 “트리 하나 대 숲 대 GBDT” 비교가 바로 그 판단의 **핵심 자료** — 어떤 데이터에 트리 하나로 끝낼 것인가, 숲을 만들 것인가, GBDT까지 갈 것인가.